Bounded Contexts and Service Boundaries

where the language changes, split

Watch the 4-min video

In one line: a bounded context is a boundary inside which one model and one language apply; contexts follow business capabilities, own their data, and are the best guide to service boundaries: high cohesion inside, few dependencies outside.

1 What is a customer?

Ask billing and a customer is a payment method and invoices. Ask support and it is tickets and contact history. Ask marketing and it is segments and campaigns. One giant `Customer` model that pleases all three grows to dozens of optional fields, and every team must coordinate to change it. DDD's answer is not one model but several small ones, each valid in its own **bounded context**.

2 A bounded context

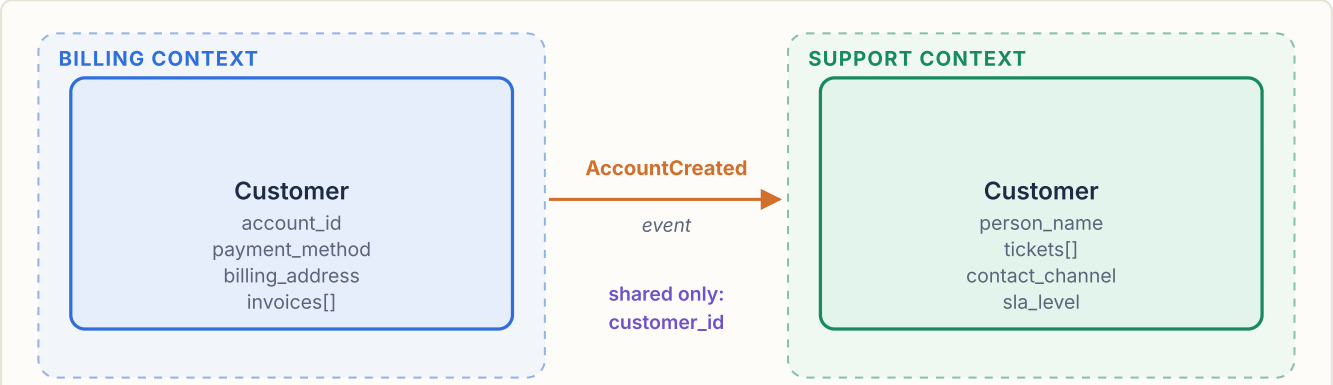


Figure 1. The same word is two concepts. Each context keeps its own model and data, linked only by an ID and events.

Inside a context, every term in the ubiquitous language (lesson 21) has one precise meaning. The same word in two contexts is really two concepts, and that is fine. Each context keeps a small, consistent model instead of a shared one that serves no one well.

3 Finding the boundaries

- **Business capabilities:** what the business does (billing, shipping, catalog, support), not how the code is layered.
- **Language changes:** where a word changes meaning, a boundary is near.
- **Team ownership:** one team should be able to own a context end to end (Conway's Law, lesson 19).
- **Data ownership:** each context owns its data; others read it through an API or events, never by writing its tables.

Talking to domain experts, or running an **event storming** workshop where the team lays out domain events on a timeline, reveals these seams quickly. Each context is a candidate service, though one service may hold several small contexts at first.

4 The context map

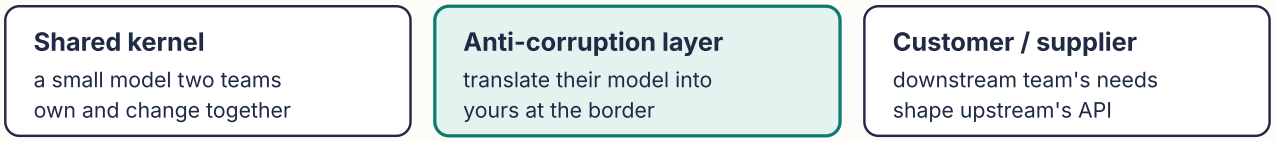


Figure 2. Common relations on a context map. Drawing them makes hidden dependencies visible.

Contexts still work together. A **context map** shows how: a **shared kernel** is a small model shared and changed jointly (use rarely); an **anti-corruption layer** translates another context's model so its concepts don't leak into yours (lesson 30); in a **customer/supplier** relation the upstream team plans for the downstream team's needs. A **conformist** simply adopts the upstream model as is.

5 Testing a boundary

Signal	Good boundary	Wrong boundary
A typical feature	changes one service	changes three services in lock-step
Calls per request	few, coarse-grained	chatty: many small calls
Data	owned by one service	two services write the same table
Split by	business capability	technical layer (a "validation service")

Two failure modes: boundaries **too fine** create chatty services that call each other on every request; boundaries **cut by technical layer** spread one business change across many teams. If two services constantly need each other's data or always change together, merge them or move the line.

WORKED EXAMPLE: BILLING VS SUPPORT

In **billing**, `Customer` = account ID, payment method, billing address, invoices. In **support**, `Customer` = person name, tickets, contact channel, SLA level. Each context stores its own customer table; the only shared value is `customer_id`. When an account is created, billing publishes `AccountCreated {customer_id, name, email}`; support consumes it and creates its own record with a default SLA level. Billing can rename or add fields freely, because support depends only on the event contract.

COMMON MISTAKES

- One shared canonical model for the whole company.
- Splitting by technical layer instead of business capability.
- Services so fine-grained that every request fans out to ten calls.
- Letting another context write your tables "to save an API".

6 Key terms

Bounded context: boundary within which one model and language apply

Context map: diagram of how contexts relate

Anti-corruption layer: translator that keeps a foreign model out

Business capability: something the business does, e.g. billing

Shared kernel: small model two teams share and change together

Event storming: workshop mapping domain events to find contexts

7 Interview questions

Q1. What is a bounded context? BASIC

A boundary within which one domain model and one ubiquitous language apply; the same word may mean something else in another context.

Q2. What is a context map? BASIC

A diagram of how bounded contexts relate: shared kernel, anti-corruption layer, customer/supplier, conformist, and which team adapts to which.

Q3. How do you decide service boundaries? SCENARIO

Follow business capabilities and bounded contexts: where language changes, where data is owned, where a team can work alone. Aim for high cohesion inside and few dependencies outside.

Q4. Billing and support both need customer data. How? SCENARIO

Each keeps its own customer model and table linked by customer_id; billing publishes AccountCreated and support builds its own record from it.

Q5. Two services always deploy together and call each other constantly. What does that tell you? TRICKY

The boundary is wrong: they are one context split in two. Merge them or move the line so most changes stay inside one service.

Q6. Is one bounded context always one microservice? TRICKY

No. A context is a modelling boundary; a service may contain several small contexts, but one context should not be split across services.

Next: SD-L-005 · Database per Service backendroadmap.com